CHAPTER 17

Debugging Ruby

Sometimes code doesn't work as you expect. Ruby provides a lot of different ways to see what's happening as you execute your code to enable you to debug and determine what is happening. From humble print statements to elaborate inline debugging tools, you can get the visibility you need into your code.

Printing Things

If you want to debug code and you don't want to use any fancy tools, then printing things out to the console is the way to go. We don't mean to make fun—we extensively use this method. It's quick and lends itself to faster cycle times than using a debugger to step through code.

We've seen a few options for this already, like puts, which uses to_s to convert its argument to a string, and p, which does the same thing but uses inspect. Ruby also provides print and pretty-print via the pp method. Here's a table of what they all look like for nil, a string, a symbol, an array, a hash, and an array with a hash element:

print	p	puts	pp
	nil		nil
test	"test"	test	"test"
test	:test	test	:test
[1, 2, 3]	[1, 2, 3]	1 2 3	[1, 2, 3]
{:a=>1, :b=>2}	{:a=>1, :b=>2}	{:a=>1, :b=>2}	{:a=>1, :b=>2}
[1, 2, {:a=>1, :b=>2}]	[1, 2, {:a=>1, :b=>2}]	1 2 {:a=>1, :b=>2}	[1, 2, {:a=>1, :b=>2}]

Table 8—print Method

There are some differences here, but they are subtle. (This is, by the way, exactly the kind of table an author puts in a book when the author can never remember the differences. It's now a convenient place for the author to look them up.)